

BIOS 735: Tree Based Classifiers

Alexander McLain

September 22, 2026

Outline

Introduction to CART

How to grow a Regression tree

How to grow a Classification tree

A Motivating Example

Suppose we want to predict whether patients are at high risk of complications after surgery.

Potential predictors include:

- ▶ Age
- ▶ BMI
- ▶ Smoking status
- ▶ Blood pressure

The important predictors may depend on the patient:

- ▶ Among younger patients, BMI may be most informative.
- ▶ Among older patients, blood pressure may be more informative.

Question

How can we allow the important predictors and their relationships to change across different groups of patients?

Limits of Standard Regression

Regression models require us to specify how predictors enter the model.

$$Y = f(X_1, \dots, X_p) + \epsilon$$

For example, we may need to specify:

- ▶ Functional forms for continuous predictors
- ▶ Interaction terms such as Age $\times$ BMI
- ▶ Higher-order interactions

As the number of predictors grows, there are many possible forms and interactions to consider.

CART

Trees avoid specifying these relationships in advance by **partitioning the predictor space**.

Why CART?

Classification and Regression Trees (CART) provide a flexible, nonparametric approach.

A tree can automatically:

- ▶ Detect nonlinear relationships
- ▶ Identify interactions and higher-order interactions
- ▶ Define different prediction rules for different subgroups

No need to specify:

functional form or interaction terms

Key idea

CART finds a set of simple rules that partition the predictor space into regions with similar outcomes.

CART: Partitioning the Predictor Space

Suppose we have two predictors, X_1 and X_2 , and want to classify observations as disease vs. no disease.

A tree partitions the predictor space into **rectangular regions**.

For example:

$$X_1 < 3.5 \quad \text{and} \quad X_2 \geq 2.1$$

defines one region.

Each region has its own prediction.

The basic strategy

Partition the predictor space $\rightarrow$ make a simple prediction within each region.

CART: Basic Idea

This strategy is **nonparametric**:

Y does not need to follow a specified functional form of X .

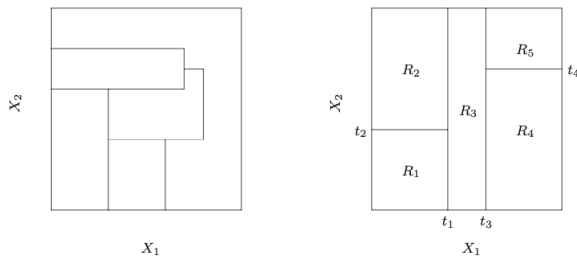
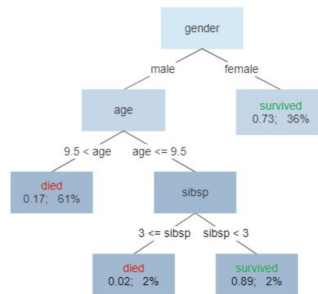


Figure: CART Example from ESL II (page 306): recursive binary splits divide the predictor space into regions.

CART Regions and Predictions

- ▶ The ends of the tree are called **terminal nodes** or terminal regions.
- ▶ Denote the terminal regions by $R_1, \dots, R_M$.
- ▶ Every observation belongs to exactly one terminal region.

Survival of passengers on the Titanic



CART Regions and Predictions

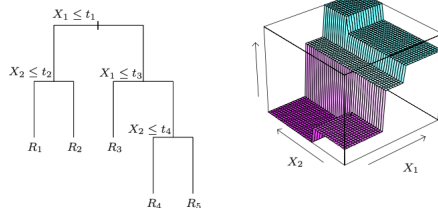


Figure: CART Example from ESL II (page 306). Predictor space is divided into decision regions. Within each terminal region, the prediction is constant.

$$f(x) = \sum_{m=1}^M c_m I(x \in R_m)$$

For regression:

$$\hat{c}_m = \text{ave}(y_i \mid x_i \in R_m)$$

What Does a Regression Tree Estimate?

Suppose we have N observations (x_i, y_i) .

We want to divide the predictor space into regions

$$R_1, \dots, R_M$$

and assign a constant prediction to each region.

$$f(x) = \sum_{m=1}^M c_m I(x \in R_m)$$

If we minimize squared error, the optimal prediction within a region is

$$\hat{c}_m = \text{ave}(y_i \mid x_i \in R_m)$$

Important

Once the regions are determined, estimating the prediction within each region is easy.

The difficult part is finding the regions.

How Do We Choose the Regions?

In principle, we could search over all possible partitions.

This quickly becomes computationally infeasible.

Number of possible trees grows extremely rapidly.

Instead, CART uses a **greedy recursive algorithm**:

1. Start with all observations in one region.
2. Consider every predictor and possible split.
3. Choose the split that gives the largest reduction in error.
4. Repeat within the resulting regions.

Greedy

At each step, CART chooses the best split available **right now**.

Choosing a Split

Consider splitting region R using predictor X_j at value s :

$$R_1(j, s) = \{X : X_j \leq s\}$$

$$R_2(j, s) = \{X : X_j > s\}.$$

CART searches over:

every predictor j and every possible split s

and chooses the split that minimizes the total within-region error.

Splitting: Optimization Problem

For squared error loss, CART chooses (j, s) to minimize

$$\min_{j,s} \left[\sum_{x_i \in R_1(j,s)} (y_i - \hat{c}_1)^2 + \sum_{x_i \in R_2(j,s)} (y_i - \hat{c}_2)^2 \right].$$

where

$$\hat{c}_1 = \text{ave}(y_i \mid x_i \in R_1), \quad \hat{c}_2 = \text{ave}(y_i \mid x_i \in R_2).$$

In practice

Scan over predictors and split points $\rightarrow$ choose the split that produces the most homogeneous child regions.

Further Splits

Suppose the first split gives

$$R_1 = \{X : X_1 \leq 4\}, \quad R_2 = \{X : X_1 > 4\}.$$

For the next split, we search **within each existing region**.

- ▶ Try every predictor.
- ▶ Try every possible split.
- ▶ Choose the best split within each current terminal region.

Thus, a tree is built by repeatedly applying the same simple operation:

Choose a region → find its best split

How Far Should We Grow the Tree?

If we continue splitting:

- ▶ Terminal nodes become smaller and smaller.
- ▶ Eventually, each observation may have its own node.
- ▶ Training error approaches zero.

But a very large tree can **overfit**.

So tree size becomes a tuning parameter.

Goal

Find a tree that balances predictive accuracy with model complexity.

Cost-Complexity Pruning

Let $C(T)$ denote the training error of tree T .

As the tree gets larger,

$C(T)$ always decreases.

So we penalize tree size:

$$C_{\alpha}(T) = C(T) + \alpha|T|$$

where $|T|$ is the number of terminal nodes.

- ▶ Large α : stronger penalty $\rightarrow$ smaller tree
- ▶ Small α : weaker penalty $\rightarrow$ larger tree

Cost-Complexity Pruning and Regularization

The idea is closely related to penalized regression.

Lasso/Ridge: $\text{fit} + \text{penalty on model complexity}$

CART: $\text{fit} + \alpha(\text{number of terminal nodes})$

In both cases:

better fit $\longleftrightarrow$ simpler model
--

Key idea

Tree pruning is a form of **regularization**.

Weakest Link Pruning

How do we find the candidate subtrees?

1. Start with the full tree T_0 .
2. Consider removing each internal split.
3. Remove the split that produces the smallest increase in training error relative to the reduction in tree size.
4. Repeat.

This produces a nested sequence of subtrees:

$$T_0 \supset T_1 \supset T_2 \supset \cdots \supset T_{\text{root}}.$$

Each subtree corresponds to a range of values of α .

Choosing the Tree Size

We now have a sequence of candidate trees.

How do we choose among them?

Cross-validation

For each candidate tree:

1. Calculate its prediction error using cross-validation.
2. Choose the tree with the best estimated out-of-sample performance.

Important

The greedy algorithm determines the candidate tree structure; cross-validation determines how much of the tree to keep.

Classification Trees

For a classification problem, $y_i \in \{1, 2, \dots, K\}$.

The tree still partitions the predictor space into terminal regions.

Within terminal node m , estimate

$$\hat{p}_{mk} = \frac{1}{N_m} \sum_{x_i \in R_m} I(y_i = k).$$

The predicted class is

$$k(m) = \arg \max_k \hat{p}_{mk}.$$

What changes?

The tree-building algorithm is essentially the same. What changes is how we measure within-node heterogeneity.

Node Impurity Measures

To choose classification splits, we need a measure of impurity.

Misclassification error: $1 - \max_k \hat{p}_{mk}$

Gini index: $\sum_{k=1}^K \hat{p}_{mk}(1 - \hat{p}_{mk})$

Entropy: $-\sum_{k=1}^K \hat{p}_{mk} \log(\hat{p}_{mk})$

All measure how heterogeneous the classes are within a node.

Intuition

A good split creates child nodes that are **purier** than the parent.

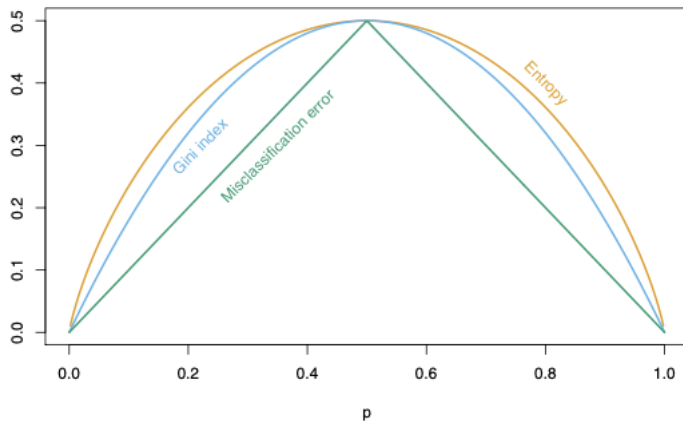


Figure: CART example of impurity measures for 2 classes as a function of p (proportion in Class 1). From ESL II (page 309).